

The Thread Abstraction

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

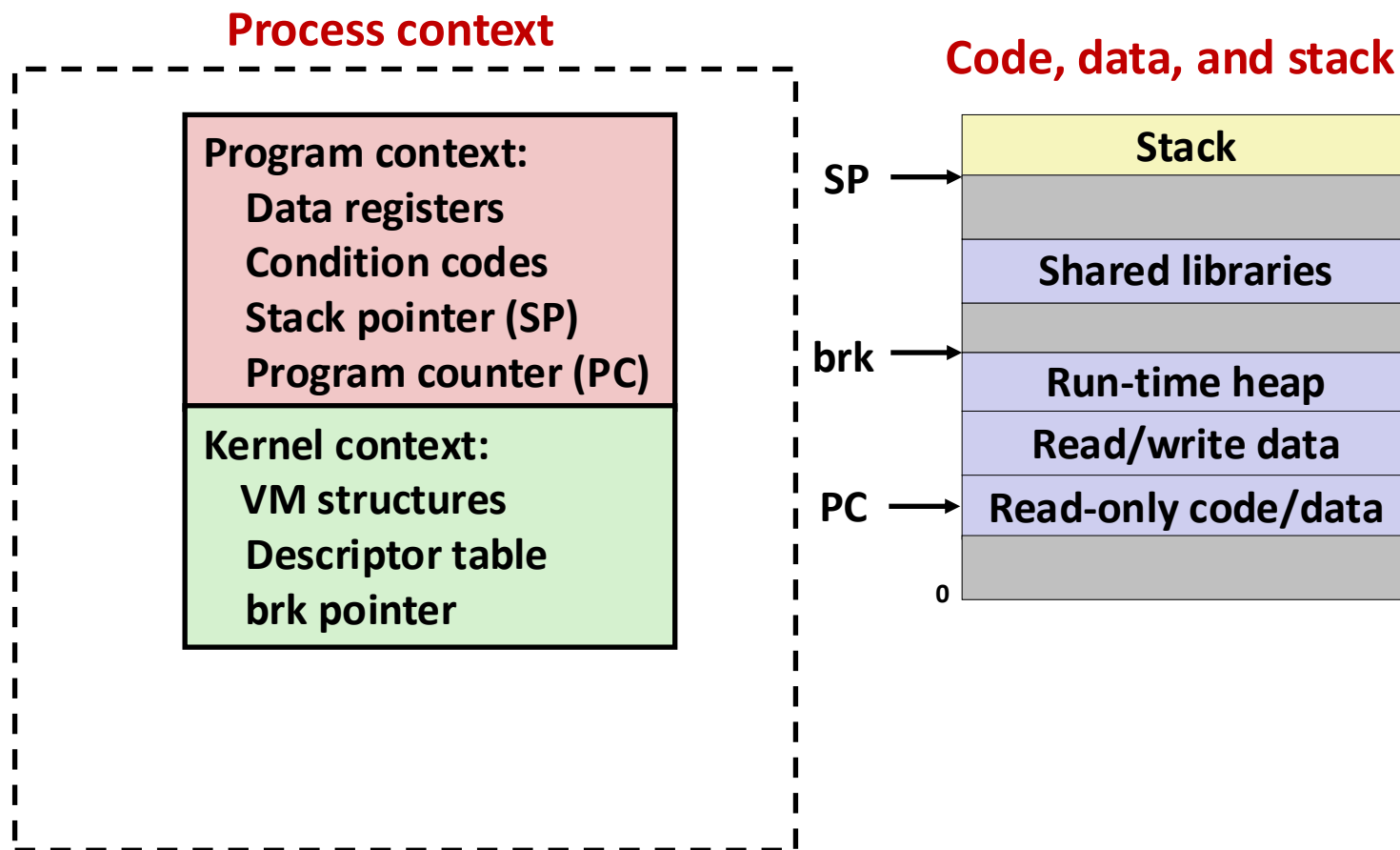
- The thread abstraction
- Sharing data
- Thread-safe functions

Processes and threads

- By default, processes have one sequential thread of execution
- OSes evolved to offer the ability to have **multiple concurrent threads** of execution in a process
 - A thread is a lightweight execution unit within a process that shares the process address space
- Requires changes to the process model
 - Must manage CPU state on a **per-thread basis**
 - All other resources can be managed on a per-process basis

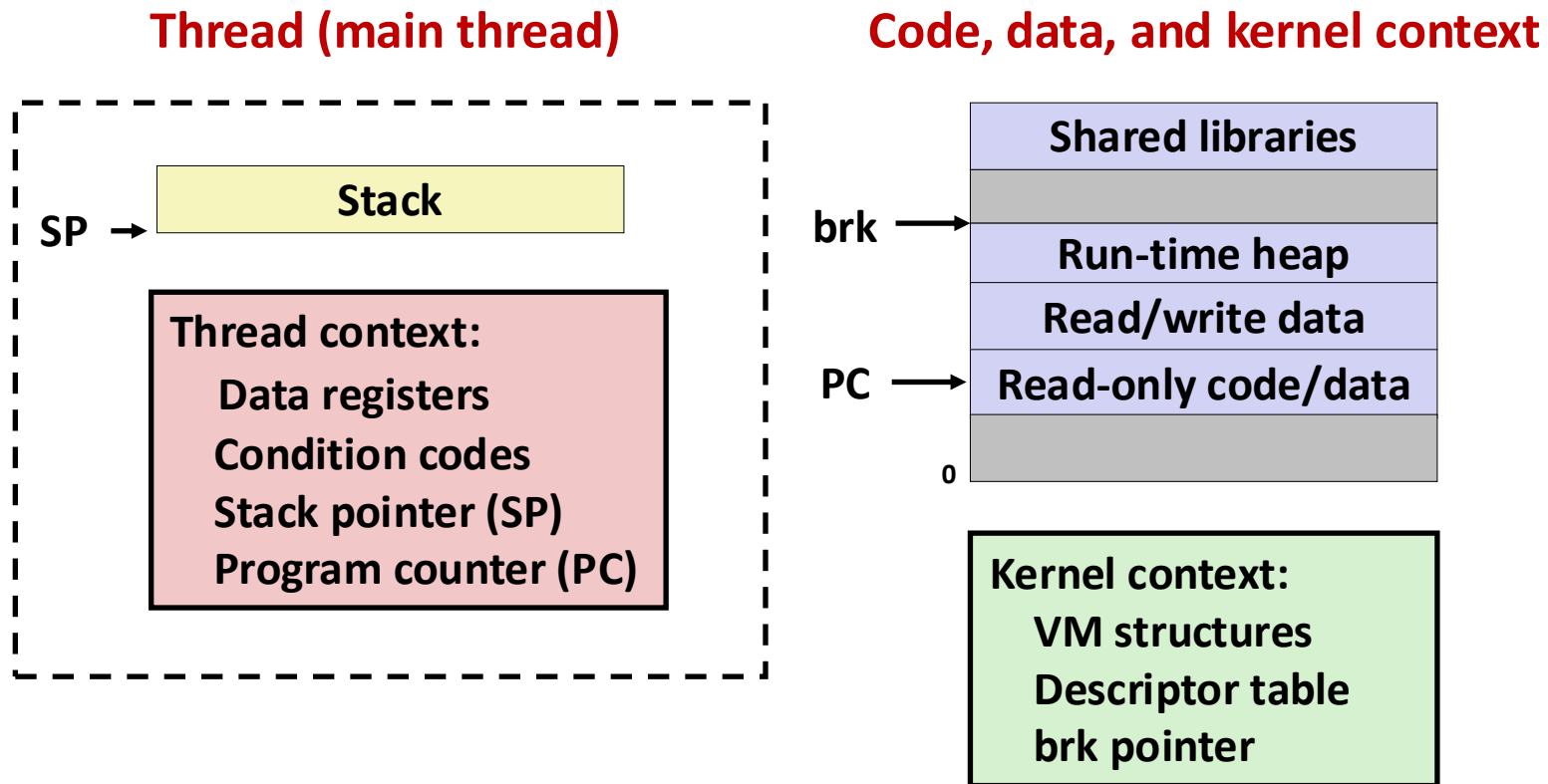
Traditional view of a process

- Process = process context + code, data, and stack



Alternate view of a process

- Process = thread + code, data, and kernel context



A process with multiple threads

■ Multiple threads can be associated with a process

- All threads share the same code, heap, global data, and kernel context
- Each thread has its own logical control flow
- Each thread has its own stack for local variables
 - But not protected from other threads!
- Each thread has its own thread ID (TID)

Thread 1 (main thread)

stack 1

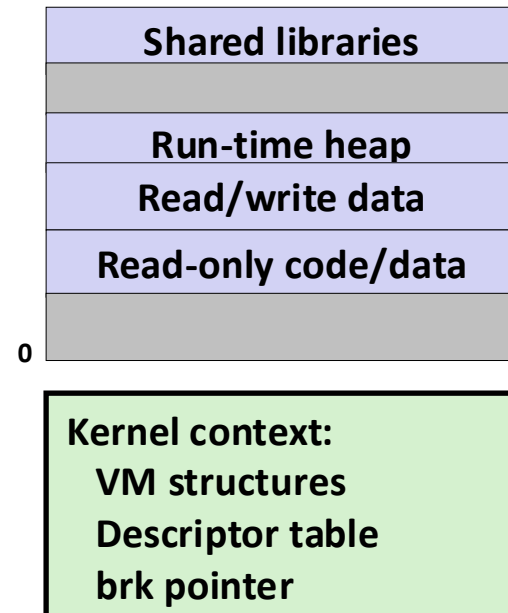
Thread 1 context:
Data registers
Condition codes
SP₁
PC₁

Thread 2 (peer thread)

stack 2

Thread 2 context:
Data registers
Condition codes
SP₂
PC₂

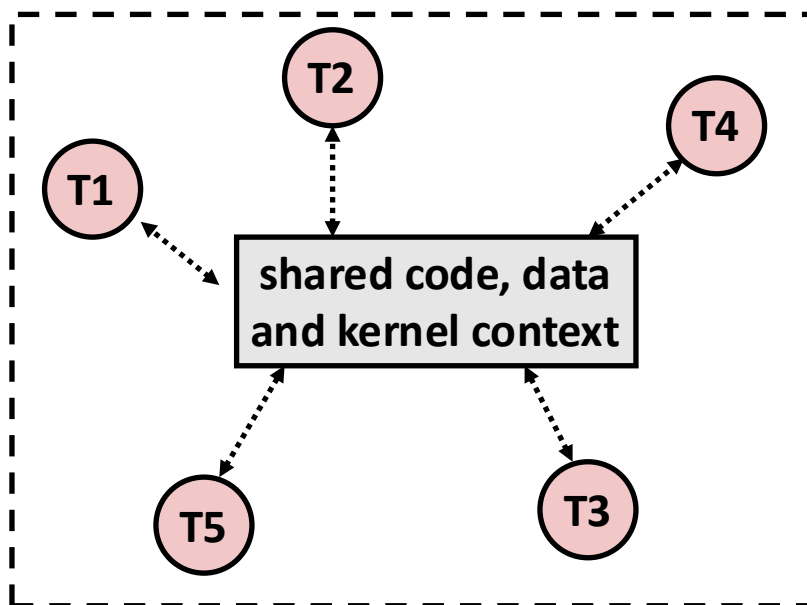
Shared code and data



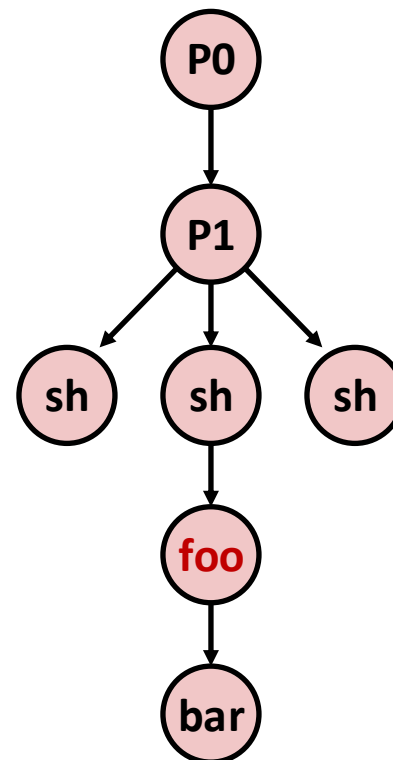
Logical view of threads

- Threads associated with a process form a **pool of peers**
 - Unlike processes which form a tree hierarchy

Threads associated with process `foo`



Process hierarchy



Why multithreaded processes?

+ Lower creation and management overhead

- Threads are faster to create and destroy than processes
- Most resources (memory, files, sockets, ...) are managed per process, not per thread

+ Faster context switching

- Switching between threads in the same process is more efficient
- Threads share the same address space; no need to switch page tables or memory mappings
- Switching between processes remains more expensive

Why multithreaded processes?

+ Simpler resource sharing

- Threads naturally share memory and process resources
- No need for complex IPC mechanisms (e.g., pipes, shared memory setup)

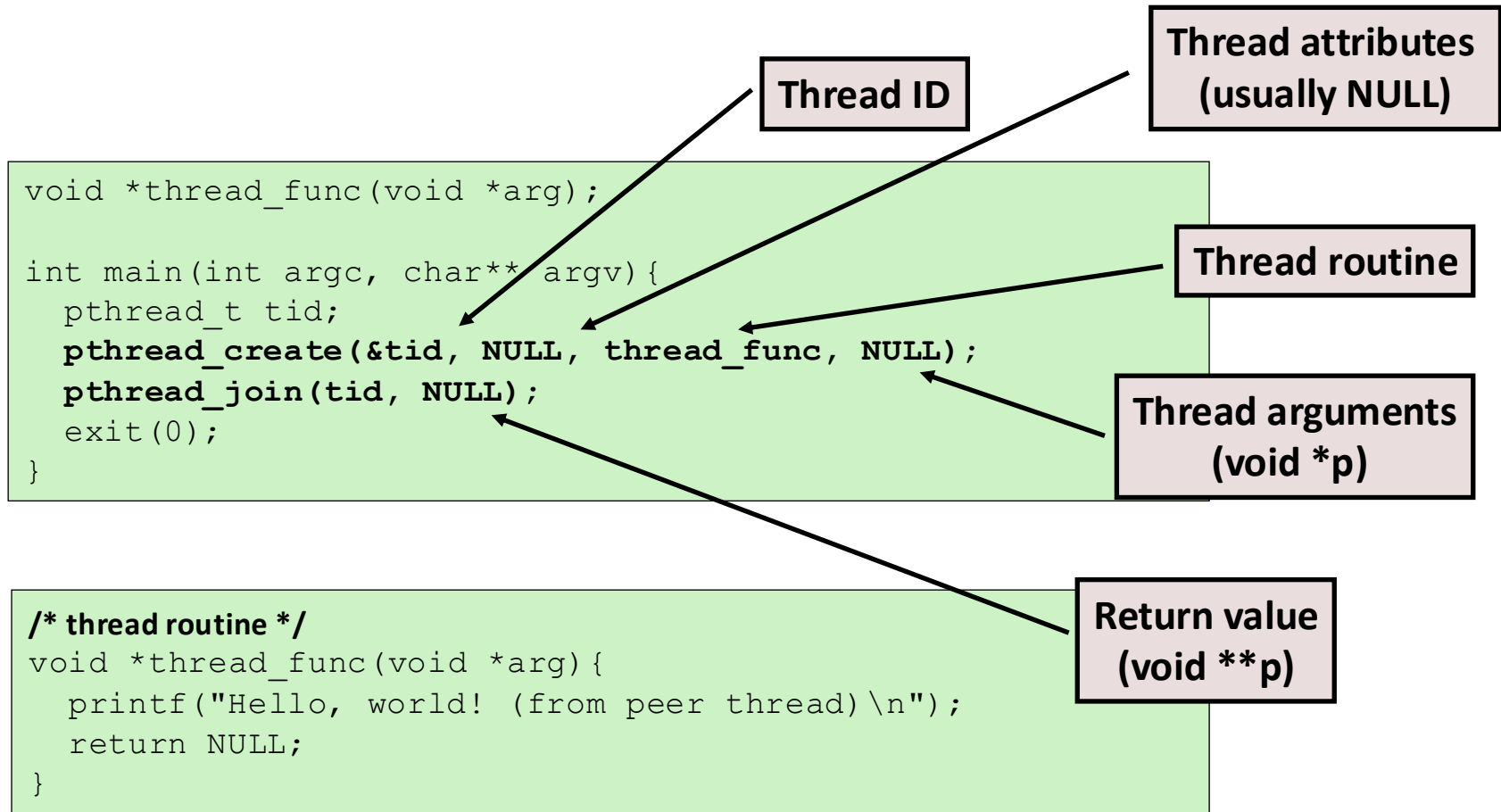
– Significantly increase complexity and risk of subtle bugs

- Threads share memory by default
- Execution order is non-deterministic
- Unsynchronized access can lead to incorrect or inconsistent results

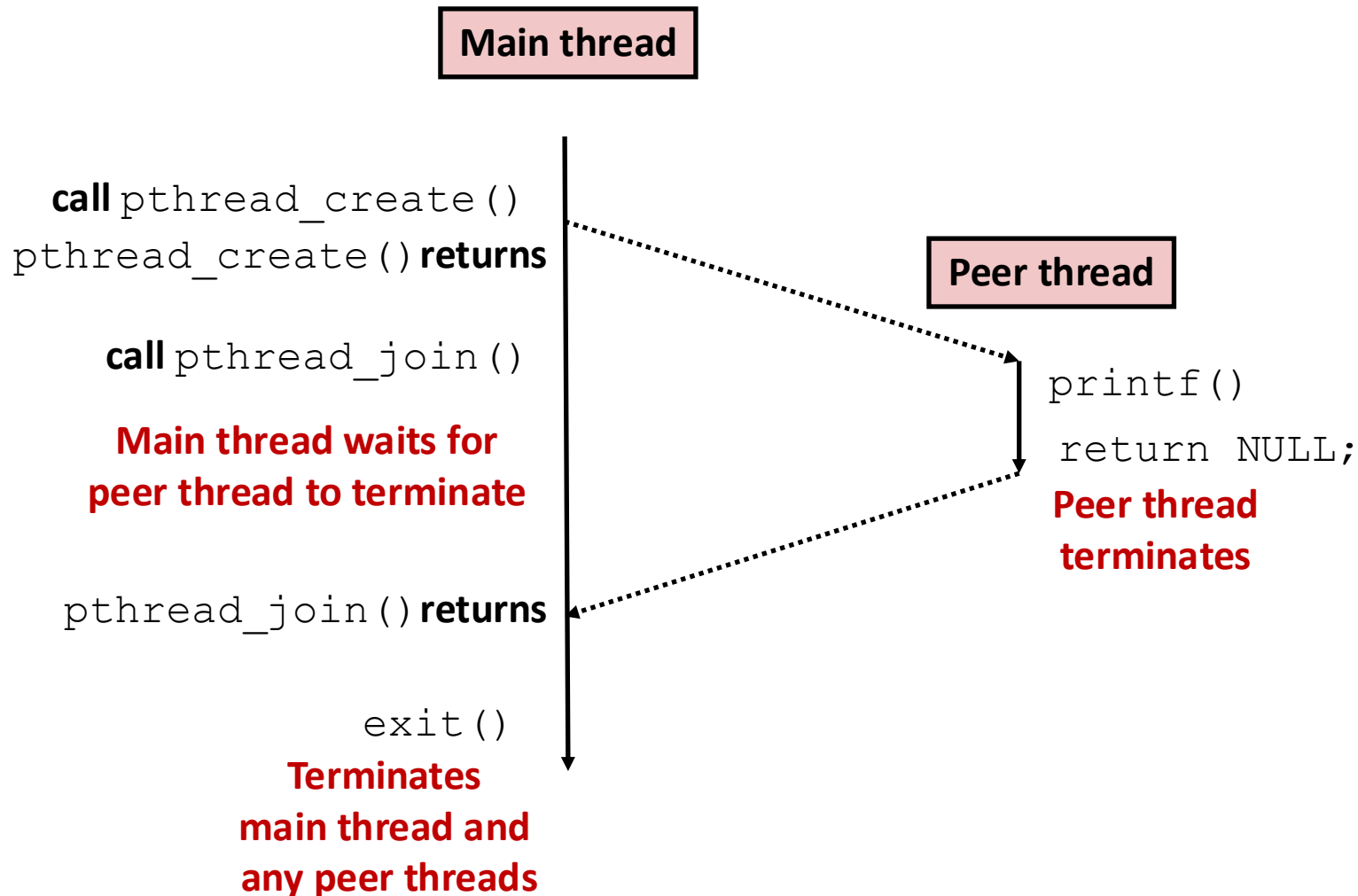
POSIX threads interface

- **Standard interface for ~60 functions that manipulate threads from C programs**
- **Thread creation**
 - `pthread_create()` spawns a new execution unit by mapping a specific function to a thread
- **Termination**
 - Execution ends when the thread routine returns or via an explicit `pthread_exit()` call
- **`pthread_join()` performs two critical roles:**
 - Wait: Blocks the caller until the target thread terminates
 - Cleanup: Reclaims system resources associated with the finished thread

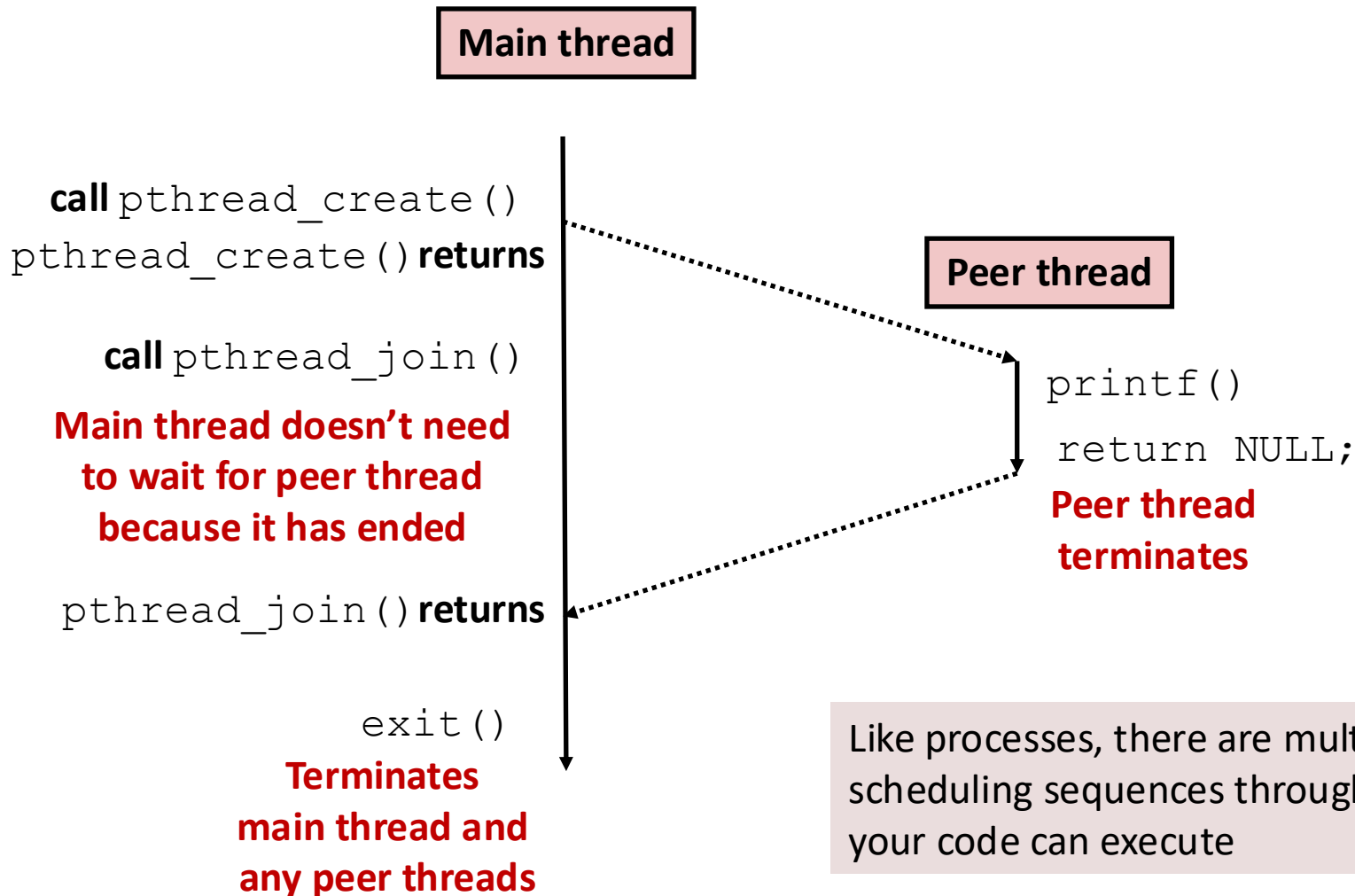
The pthreads "hello, world" program



Execution of threaded “hello, world”



Or, ...



Overview

- The thread abstraction
- **Sharing data**
- Thread-safe functions

Shared variables in threaded C programs

■ Which variables in a threaded C program are shared?

- The answer is not as simple as “*global and heap variables are shared*” and “*stack variables are private*”

■ A variable **x** is shared **if and only if** multiple threads reference some instance of **x**

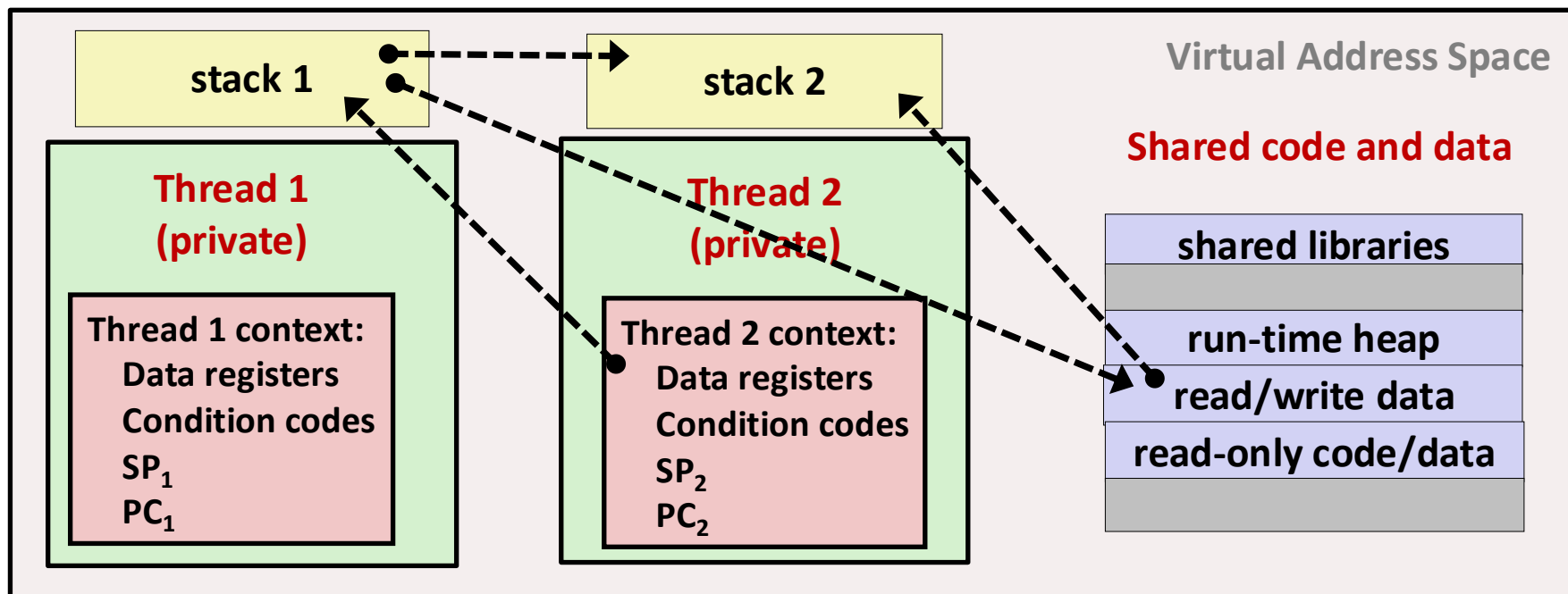
■ Requires answers to the following questions:

- What is the memory model for threads?
 - (more on this in the next lecture)
- How are instances of variables mapped to memory?
- How many threads might reference each of these instances?

Threads memory model

■ Separation of data is not strictly enforced

- Register values are truly separate and protected, but...
- ...any thread can potentially read and write the stack of any other thread
- **The mismatch between the conceptual and operation model is a source of confusion and errors**



Mapping variable instances to memory

■ Global variables

- *Def*: Variable declared outside of a function
- **Virtual memory contains exactly one instance of any global variable**

■ Heap variables

- *Def*: Variable dynamically reserved at runtime in the heap
- **Virtual memory contains exactly one instance of any heap variable**

■ Local variables

- *Def*: Variable declared inside function without the **static** attribute
- **Each thread stack contains one instance of each local variable**

Mapping variable instances to memory

■ Local static variables

- *Def*: Variable declared inside function with the **static** attribute
- **Virtual memory contains exactly one instance of any local static variable**

■ `errno` is special

- To ensure thread safety, `errno` is implemented as a **thread-local variable**
- Provides each thread with its own private instance to store error codes without interference from peers

Mapping variable instances to memory

```
char **ptr;

int main(int main, char *argv[]){
    int i;
    pthread_t tid;
    char *msgs[2] = {
        "Hello from foo",
        "Hello from bar"
    };

    ptr = msgs;
    for (i = 0; i < 2; i++){
        pthread_create(&tid, NULL,
                      thread_func,
                      (void *)&i);
    }
    pthread_exit(NULL);
}
```

```
void *thread_func(void *arg){
    int myid = *((int*)arg);
    static int cnt = 0;

    printf("[%d]: %s (cnt=%d)\n",
           myid, ptr[myid], ++cnt);
    return NULL;
}
```

Mapping variable instances to memory

Global var: 1 instance (`ptr` [data])

Local vars: 1 instance (`i.m`, `msgs.m`, `tid.m`)

```
char **ptr;

int main(int main, char *argv[]){
    int i;
    pthread_t tid;
    char *msgs[2] = {
        "Hello from foo",
        "Hello from bar"
    };

    ptr = msgs;
    for (i = 0; i < 2; i++){
        pthread_create(&tid, NULL,
                      thread_func,
                      (void *)&i);
    }
    pthread_exit(NULL);
}
```

Local var: 2 instances
(`myid.t0` [peer thread 0's stack],
`myid.t1` [peer thread 1's stack])

```
void *thread_func(void *arg){
    int myid = *((int*)arg);
    static int cnt = 0;

    printf("[%d]: %s (cnt=%d)\n",
           myid, ptr[myid], ++cnt);
    return NULL;
}
```

Local static var: 1 instance (`cnt` [data])

Shared variable analysis

■ Which variables are shared?

Variable instance	Referenced by main thread?	Referenced by peer thread 0?	Referenced by peer thread 1?
<code>ptr</code>	yes	yes	yes
<code>cnt</code>	no	yes	yes
<code>i.m</code>	yes	yes	yes
<code>msgs.m</code>	yes	yes	yes
<code>myid.t0</code>	no	yes	no
<code>myid.t1</code>	no	no	yes

■ A variable **x** is shared if and only if multiple threads reference at least one instance of **x**. Thus:

- `ptr`, `cnt`, `i`, and `msgs` are shared
- `myid` is **not** shared

Shared variables in threaded C programs

- **Shared variables are handy, but you risk data races and synchronization errors without proper care**
 - A data race occurs when two or more threads access the same variable concurrently without proper synchronization and at least one access is a write
- **Can a race condition occur in this example?**
 - If no race, then each thread would get different value of `i`
 - Set of saved values would consist of one copy each of 0 through 99

Main

```
int i;

for(i = 0; i < 100; i++){
    pthread_create(&tid, NULL,
                  thrd_function, &i);
}
```

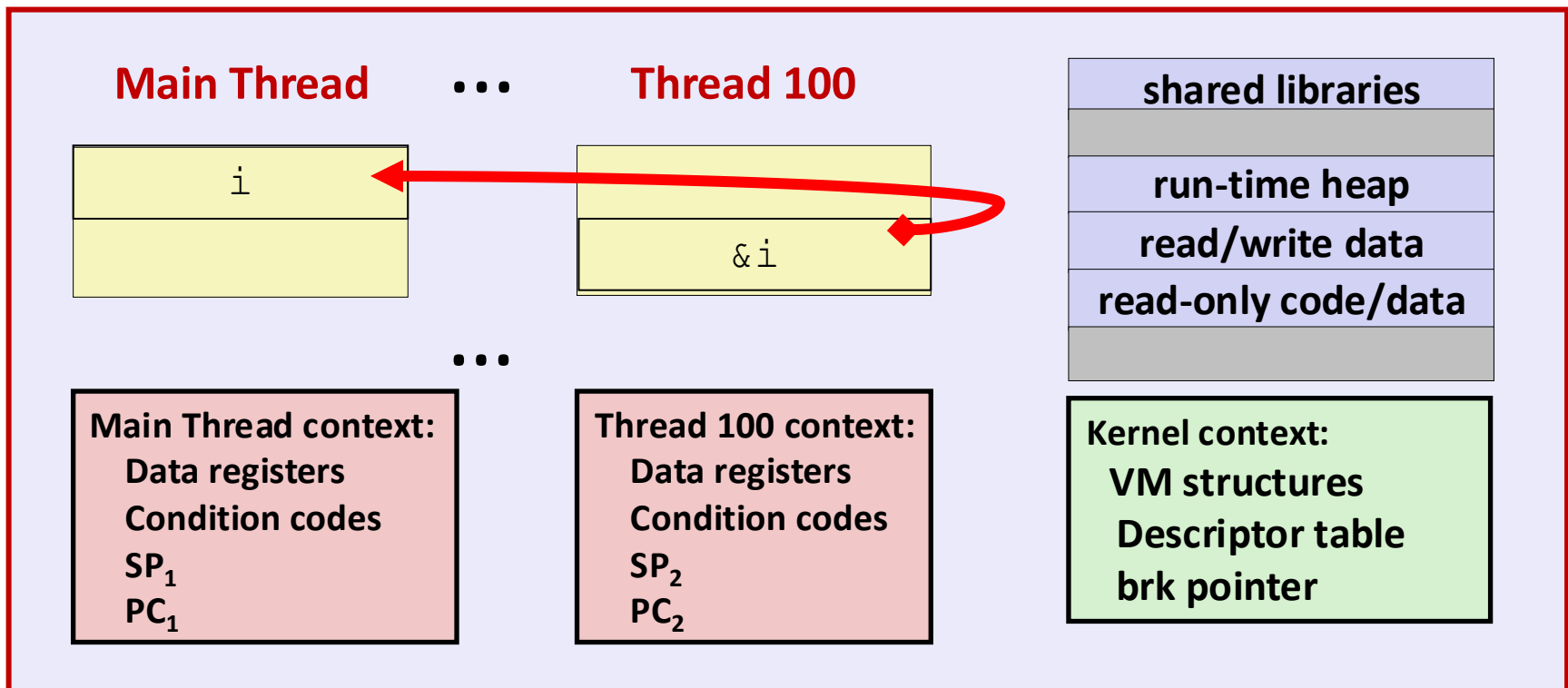
Thread t0..t99

```
void *thrd_function(void *arg){
    int i = *((int*)arg);

    save_value(i);
    return NULL;
}
```

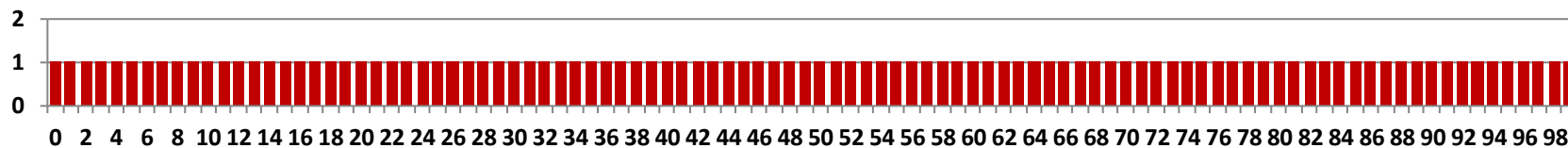
All threads receive the same memory address

```
for (i = 0; i < 100; i++){
    pthread_create(&tid, NULL, thrd_f, &i);
}
```

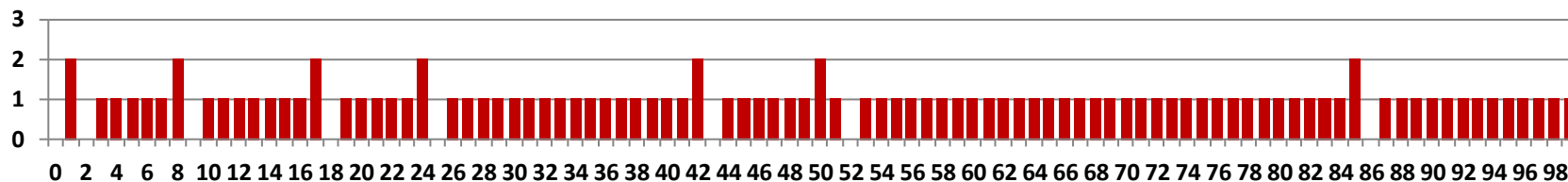


Experimental results

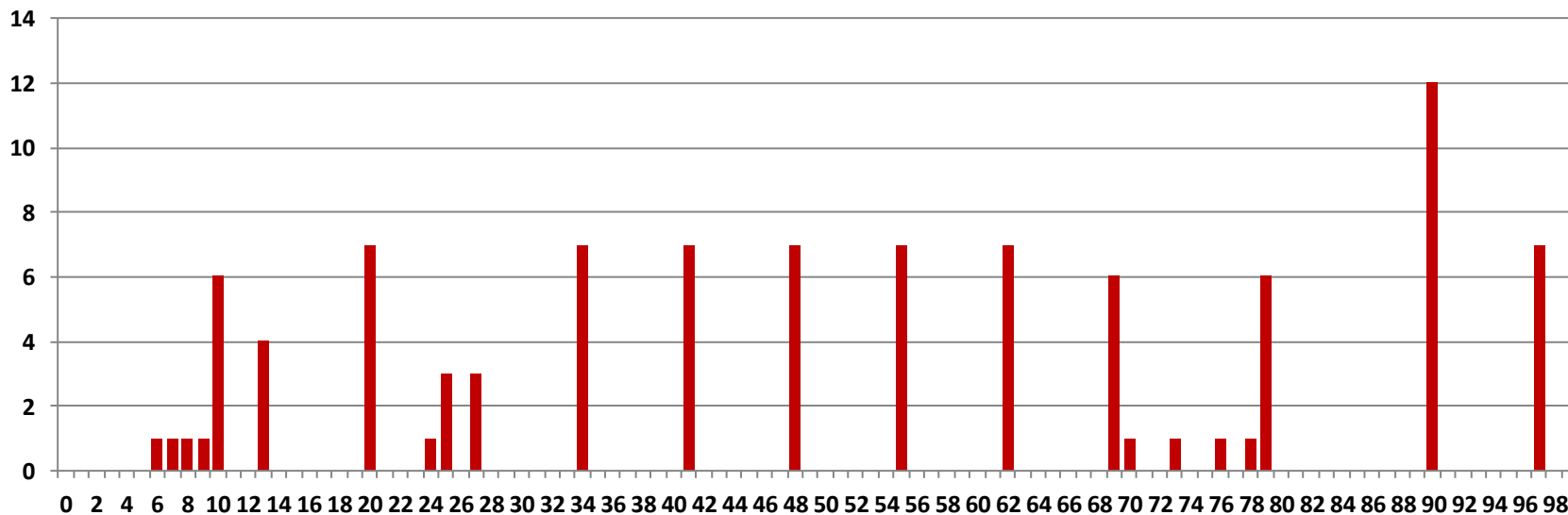
No race



Single core



Multicore



Correct passing of thread arguments

```
int hist[N] = {0};

int main(int argc, char *argv[]){
    int i;
    pthread_t tids[N];

    for (i = 0; i < N; i++){
        int* p = malloc(sizeof(int));
        *p = i;
        pthread_create(&tids[i], NULL,
                      thread_function,
                      (void *)p);
    }

    for (i = 0; i < N; i++)
        pthread_join(tids[i], NULL);

    check();
}
```

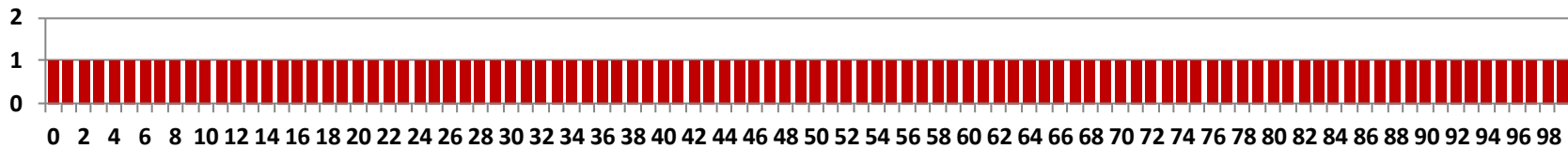
```
void *thread_function(void *arg){
    hist[(int*)arg] += 1;

    free(arg) ;
    return NULL;
}
```

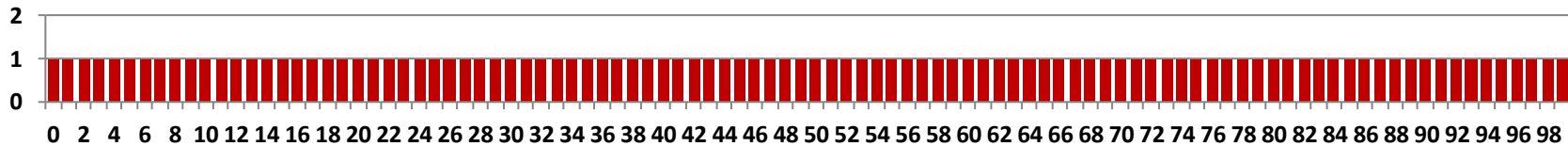
```
void check(void) {
    for (int i=0; i<N; i++) {
        if (hist[i] != 1) {
            printf("Failed at %d\n", i);
            exit(-1);
        }
    }
    printf("OK\n");
}
```

No more race condition!

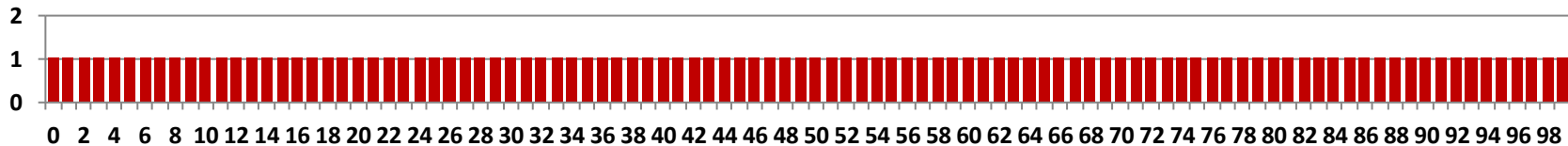
No race



Single core



Multicore



It's not only “use the heap”!

■ What does the following program print?

```
void *helper(void *arg){
    char *message = (char*)arg;
    strcpy(message, "I am the helper thread");
    return NULL;
}

int main(){
    pthread_t thread;
    char *message = malloc(100);
    strcpy(message, "I am the main thread");

    pthread_create(&thread, NULL, helper, message);

    printf("%s\n", message);

    pthread_join(thread, NULL);
    free(message)
    return 0;
}
```

Key takeaway

- **Correctness is not only about where data is allocated**
 - Stack vs heap vs global is not enough to determine safety
- **What really matters: how data is passed and accessed**
 - If multiple threads referencing the same memory location, there is a potential race condition
 - Even stack variables can become shared if their address is passed
- **Sharing is determined by aliasing, not just allocation**
 - Always ask: “How many threads can access this exact memory location?”

Overview

- The thread abstraction
- Sharing data
- **Thread-safe functions**

Thread-safe APIs

- A function is thread-safe if it always produces correct results, even when called by multiple threads at the same time
- Thread safety depends on **what is shared** and **how it is accessed**
 - Not all bugs come from obvious shared variables
- **Common source of problems:**
 - Shared global or static variables
 - Passing the same data to multiple threads
 - Hidden internal state inside functions

Thread-safe APIs

■ Writing correct multithreaded code is harder than it looks

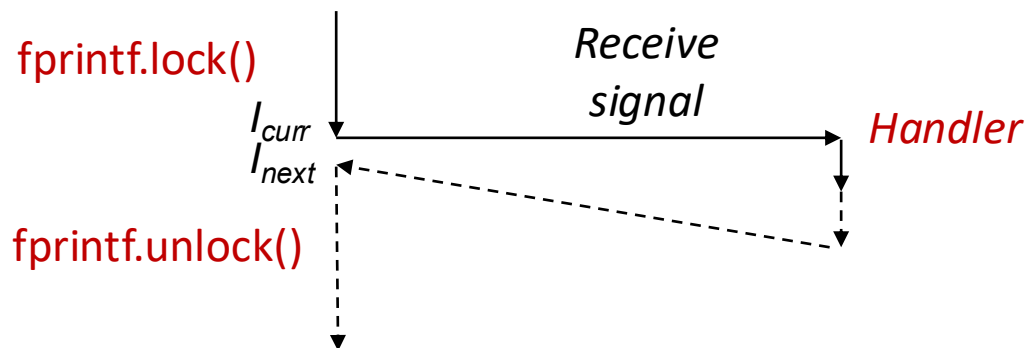
- Small mistakes can lead to subtle and unpredictable bugs

■ How to make code safer:

- Avoid sharing data when possible
- Give each thread its own copy of data
- Be careful when passing pointers
- If sharing is necessary, use synchronization mechanisms

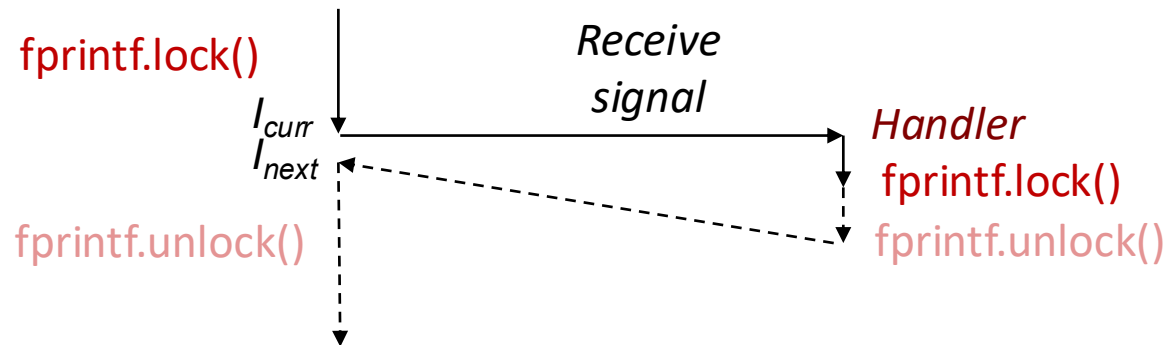
Threads/signals interactions

- Many library functions achieve thread safety using lock-and-copy
 - Internal locks protect shared state
 - Temporary data is copied to avoid interference between threads
- E.g., `fprintf` prevents interleaved output from multiple threads
 - Safe to interrupt execution



Threads/signals interactions

- This is safe only if the signal handler does not call the same (or related) functions!
 - Signal handler cannot proceed until it gets lock
 - Main program cannot proceed until handler completes
 - Deadlock!



Thread-safe library functions

■ Most ISO C library functions are thread-safe

- Examples: `malloc`, `free`, `printf`, `scanf`
- Exceptions: `strtok`, `rand`, `asctime`, ...

■ Many older Unix C library functions are unsafe

- There is usually a safe replacement

Thread-unsafe function	Class	Reentrant version
<code>asctime</code>	3	<code>strftime</code>
<code>ctime</code>	3	<code>strftime</code>
<code>localtime</code>	3	<code>strftime</code>
<code>gethostbyname</code>	3	<code>getaddrinfo</code>
<code>gethostbyaddr</code>	3	<code>getnameinfo</code>
<code>inet_ntoa</code>	3	<code>getnameinfo</code>
<code>rand</code>	2	<code>rand_r*</code>

** The C library's random number generators are all old and not very "strong". Use a modern CSPRNG instead.*

Thread safety

- **A function being thread-safe does not mean it is safe in every situation**
 - Be careful when combining threads, library functions, and signal/interrupt handlers
- **Avoid calling non-reentrant functions inside signal handlers**
 - Prefer reentrant functions, or minimal, safe operations (e.g., setting flags)
- **Thread-safe ensures correctness under concurrency**
 - Reentrancy is required for interrupt-safe execution

Reentrant functions

- **A reentrant function can be interrupted during execution and safely called again, without corrupting its internal state**
 - This may happen with multiple threads, or due to interrupts/signal handlers

- **Reentrant functions do not rely on external or persistent state**
 - They behave like pure, independent executions
 - Does not read or write global or static variables
 - No hidden state between calls; each invocation is independent
 - Uses local variables, memory provided by the caller, or thread-local storage

Reentrant vs thread-safe

- **All reentrant functions are thread-safe**
 - Not all thread-safe functions are reentrant
- **Thread-safe functions may use synchronization mechanisms**
 - Risk of deadlock if re-entered
- **Reentrant functions are a stronger guarantee than thread-safe**
 - Have no hidden state and do not require locks or synchronization
 - Inherently safer in concurrent and interrupt contexts
- **Refer to the example provided on Moodle for a hands-on demonstration of how these concepts are applied in practice**

Summary

- **Concurrency improves performance and responsiveness**

- A concurrent program consists of multiple logical flows that overlap in time

- **But correctness depends on careful data sharing, proper synchronization, and safe function design**

- Without proper synchronization there is the potential for race conditions, incorrect results, and unpredictable behavior

- **Code executed by multiple threads must be thread-safe**

- Guarantee correct results and consistent shared state regardless of how threads are interleaved
- Focuses on multiple threads accessing data simultaneously (often uses locks)

Summary

- **A function is reentrant if it can be interrupted in the middle of execution and safely called again (“re-entered”) before the first invocation finishes**
 - It only operates on local data and arguments provided by the caller
 - Focuses on a single thread (or multiple) being able to “overlap” execution of the same function safely without side effects

- **All reentrant functions are thread-safe, but not all thread-safe functions are reentrant**
 - E.g., a function using a mutex is thread-safe but may not be reentrant